Internet of Things: Protocols and System Software

Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas
Universidad de Granada

Internet of Things: Protocols and System Software

Los Del DGIIM, losdeldgiim.github.io

Arturo Olivares Martos

Granada, 2026

Índice general

1. Introduction

In the current technological world, there are three different main trends:

1. Computing: computers are getting smaller, cheaper and more powerful.
2. Sensing: there is a huge variety of sensors that can be used to measure different physical properties. They are also getting smaller and cheaper.
3. Transmission: two factors are being taken into account:
   - There are more and more wirelesscommunication technologies, with different characteristics. This leads to omnipresent connectivity.
   - Energy consumption is a key factor in the design of these technologies, as many IoT devices are battery-powered. This is leading to the development of low-power communication protocols and energy-efficient hardware.

In this context, the Internet of Things (IoT) is emerging as a paradigm where everyday objects become smart and connected to the internet.

Definición 1.1 (IoT Thing)**.** An IoT thing is a physical object that is augmented with embedded electronics, making it therefore "smart".

1.1. Digital Twin

The Digital Twin is a global representation of the world, where every physical object has a digital counterpart. This means that every physical object is represented in the digital world, and every change in the physical world is reflected in the digital world and vice versa. This is a very ambitious vision, and it is still far from being achieved. However, it is a useful concept to understand the potential of IoT and the challenges that it poses. Its three main features are:

- Live: the digital twin is continuously updated with data from the physical world.
- Predictive: the digital twin can be used to predict the behavior of the physical world using simulations and machine learning techniques.
- Interactive: depending on the predictions made by the digital twin, it can change and therefore make changes in the physical world.

In this situation, the world would become observable and programmable in real time, which would have a huge impact on many different fields, such as healthcare, transportation, manufacturing, etc. However, there are still many challenges that need to be overcome to achieve this vision.

1.2. IoT Views

There are two different views on IoT:

- IoT as a network of connected *things*, aka *M2M (Machine to Machine)*.

  It is quite a low-level technical view, where the focus is on the devices and the communication between them. Internet is just seen as a communication medium, and the main goal is to connect as many things as possible.

  It is the view that is usually taken by engineers and computer scientists.

- IoT as a network of connected *data*, aka *Big Data*.

  It is a more high-level view, where there are still things but the focus is on the data that is generated by these things and how it can be used to create value. Internet is seen as a platform for data collection and analysis, and the main goal is to extract insights from the data.

  It is the view that is usually taken by business people and data scientists.

2. IoT Systems

An IoT system consists of all the HW and SW components that are required to implement an IoT application. In order to reduce the complexity of an IoT system, developpers tend to reuse the same components across different applications, and here is where the concept of *IoT platforms* comes into play.

2.1. IoT Platforms

An IoT platform is a framework that provides a set of tools and services to facilitate the development, deployment, and management of IoT applications. By using an IoT platform, developers can focus on the application logic rather than dealing with the complexities of the underlying infrastructure. There are many IoT platforms available in the market, each with its own strengths and weaknesses. There are two main categories of IoT platforms:

- Horizontal: These are general-purpose platforms that can be used for a wide range of IoT applications. They provide a broad set of features and services that can be customized to fit the specific needs of different applications.

- Vertical: These platforms are designed for specific industries or use cases. They provide specialized features and services that are tailored to the requirements of a particular domain.

2.2. IoT System Architecture

An IoT system architecture defines whole IoT system. Regarding the HW, it consists of 4 different types of components:

1. IoT Things: Also known as Smart Objects, these are the physical devices that actually interact with the environment.

2. Gateways: These are devices that act as intermediaries between the IoT Things and the Internet. Given that the IoT Things are often resource-constrained and may not have the capability to connect directly to the Internet, IoT Gateways provide the necessary connectivity and protocol translation.

3. Internet: This is the communication infrastructure that allows the IoT Gateways to connect and exchange data with other devices and services.

4. Online Services: These are cloud-based platforms and services that provide different functionalities to support the IoT applications.

However, SW components of an IoT system are also crucial to the overall functionality and performance of the system. There are three main aspects from the SW perspective that affect the whole IoT system.

2.2.1. SW Functional Blocks

From the SW perspective, it is essential to identify the functional blocks that are required to implement an IoT application. It is difficult to define a standard set of functional blocks that can be applied to all IoT applications, but for the aim of this course, we will consider three functional blocks:

1. IoT Networking: SW necessary to enable communication between all the components of the IoT system.

2. IoT Data Management: SW responsible for measuring, collecting, storing, and making data available for querying and analysis.

3. IoT Data Processing: SW responsible for analyzing the data collected from the IoT Things and extracting useful insights and information.

   There are two main recipient groups of the data in order to make use of it:

   - Humans: The data is presented to the users in a human-readable format, such as dashboards, reports, or visualizations.
   - Algorithms: The data is used as input for machine learning models, predictive analytics, or other algorithms that can make decisions or take actions based on the data.

   There are four main levels of data processing that can be applied to the data collected from the IoT Things:

   a) Descriptive: The data is only used to observe and describe the current state of the system.

   b) Diagnostic: The data is used to identify the causes of certain events or conditions in the system and explain why they happened.

   c) Predictive: The data is used to anticipate future events or conditions in the system based on historical data and patterns.

   d) Prescriptive: The data is used to perform actions or make decisions based on the insights derived from the data analysis. This has several legal and ethical implications, and usually requires human supervision and intervention to ensure that the actions taken are appropriate and that someone is legally responsible for the consequences of those actions.

2.2.2. SW Blocks Placement

The SW functional blocks can be executed in different specific HW components within the IoT system, depending on the specific requirements and constraints of the application. There are three main approaches for the placement of the SW functional blocks.

1. Cloud-based IoT Systems: The "classic" approach.

   In this approach, the biggest part of the three SW functional blocks are executed in the cloud.

   - IoT Things are responsible for collecting data and sending it to the gateway. Therefore, they only execute a small part of the IoT Networking and IoT Data Management blocks.
   - IoT Gateways are responsible for receiving the data from the IoT Things and forwarding it to the cloud. Therefore, they execute a small part of the IoT Networking.
   - The online services in the cloud are responsible for executing the majority of the three SW functional blocks, including IoT Networking, IoT Data Management, and IoT Data Processing.

2. Edge-based IoT Systems: The "modern" approach.

   This approach is similar to the cloud-based approach, but with a significant shift of the SW functional blocks towards the gateways. Given that the online services have limited resources and are under high load, it is necessary to offload some of the processing tasks to the gateways in order to reduce latency and improve performance.

3. Mesh-based IoT Systems: The "future" approach.

   In this approach, the SW functional blocks are even for shifted towards the IoT Things, which are becoming more powerful and capable of executing more complex tasks. This approach just considers gateways and online services as fallback options for the IoT Things in case they are not able to execute certain tasks.

2.2.3. SW Blocks Communication

The SW functional blocks need to communicate with each other in order to exchange data and coordinate their actions. This communication can be achieved through different connectivity options and protocols, depending on the specific requirements and constraints of the application. We will explore them in more detail in the rest of the course.

3. IoT Internetworking

During this chapter, we will focus on the IoT internetworking aspect, which is crucial for enabling communication and data exchange between different components of an IoT system. A reader may initially think that using the Internet and the well-known TCP/IP protocol suite would be sufficient for enabling communication in an IoT system, and it is in fact a good approach as these protocols have proven to be extremely successful and widely adopted in the traditional Internet. However, there are several challenges and limitations that need to be addressed when using the Internet and TCP/IP protocols in the context of IoT systems, as an usuall Internet device is very different from an IoT device in terms of resources, capabilities, and requirements. Some of these challenges and limitations include:

- Data Item Size: IoT devices often generate and exchange small data items, such as sensor readings or control commands, while the Internet and TCP/IP protocols are designed to handle larger data items, such as web pages or multimedia content.
- Not Optimized for Energy Efficiency: IoT devices are often battery-powered and have limited energy resources, while the Internet and TCP/IP protocols are not optimized for energy efficiency and may require more power consumption than what is feasible for IoT devices.
- Large Buffer Requirements: The Internet and TCP/IP protocols may require larger buffer sizes than what is available on IoT devices, which can lead to performance issues and increased latency.
- Pull vs. Push Communication: The Internet and TCP/IP protocols are primarily designed for pull-based communication, where clients request data from servers, while IoT systems often require push-based communication, where devices need to send data to other devices or applications without waiting for a request.

In order to address these challenges and limitations, there are several IoT-specific protocols and technologies that have been developed to enable efficient and effective communication in IoT systems. There are two main approaches to connect the IoT Things to the Internet:

- Application layer proxy:

  There is a proxy (usually implemented in the IoT gateway) that translates between the protocols used by the IoT Things and the protocols used by the Internet.

- The IoT Thing communicates with the proxy using dedicated IoT protocols that have nothing to do with the Internet protocols.
- The proxy stores the data received from the IoT Things and makes it available to the Internet using standard Internet protocols.

An example of this approach is when an user has to install a specific app on their smartphone in order to interact with a smart home device, such as a smart thermostat or a smart light bulb.

- IP Router:

  In this approach, the IoT Things directly use IP, but with some optimizations and adaptations to address the challenges and limitations mentioned above. An usual router may be combined (or may replace) with the IoT gateway to enable the IoT Things to connect to the Internet using IP. That way, the IoT Things can communicate directly with other devices and applications on the Internet without the need for a proxy, which is much more scalable. There is no need for specific apps or software to interact with the IoT Things, as they can be accessed using standard Internet protocols and interfaces.

During the rest of the course, we will explore in more detail the second approach, which is the one that is becoming more and more popular in the IoT ecosystem, as it provides better scalability and interoperability.

3.1. IEEE 802.15.4

IEEE 802.15.4 is a standard for low-rate wireless personal area networks (LR-WPANs) that is widely used in IoT applications. Its main focus is on providing low-power, low-cost and low-complexity wireless communication for IoT devices. It defines the Physical Layer (PHY) and part of the Data Link Layer (MAC). In addition, it defines multiple variants for each of the layers, which can be used in different scenarios and applications depending on the specific requirements and constraints of the IoT system.

Communication using this standard is based on a IEEE 802.15.4 PAN, which is a collection of devices that are connected together using the IEEE 802.15.4 standard. The members of a PAN can either be:

- Full Function Devices (FFDs): These devices can communicate with any other device in the PAN, including other FFDs and Reduced Function Devices (RFDs). They can also take over network management activities.

  If two devices need to communicate with each other but they are not in range of each other, they can use an FFD to forward messages between them, which allows for multi-hop communication and extends the range of the PAN. Only FFD have this capability.

- Reduced Function Devices (RFDs): These resource-constrained devices can only communicate with one FFD in the PAN, and they cannot take over network management activities.

In addition, each of the members of a PAN can have different roles, which determine their behavior and responsibilities in the network:

- PAN Coordinator: This is the device that is responsible for managing the PAN, including tasks such as the PAN creation (by selecting a PAN ID and a channel), adding and removing devices from the PAN, and coordinating the communication between devices in the PAN. There can only be one PAN Coordinator in a PAN, and it must be an FFD.
- Coordinator: This is a device that is responsible for managing a portion of the PAN, such as a cluster of devices. It can also take over some of the network management activities, such as routing and forwarding of messages. There can be multiple Coordinators in a PAN, and they must be FFDs.
- Members: These are the devices that are part of the PAN and can communicate with other devices in the PAN. They can be either FFDs or RFDs, and they do not have any specific responsibilities or roles in the network.

Once the members of a PAN have been defined and assigned their roles, they can communicate with each other using one of these two types of topologies:

- Star Topology: In this topology, all the devices in the PAN communicate directly with the PAN Coordinator, which acts as a central hub for the communication. This topology is simple and easy to manage, but it can be less efficient and less scalable than other topologies, as all the communication has to go through the PAN Coordinator, which can become a bottleneck and a single point of failure.
- Peer-to-Peer Topology: In this topology, the devices in the PAN can communicate directly with each other without the need for a central hub. This topology is more efficient and scalable than the star topology, as it allows for direct communication between devices, which can reduce latency and improve performance. However, it can be more complex to manage, as it requires more coordination and synchronization between devices to ensure that messages are delivered correctly and efficiently.

It should however be noted that the IEEE 802.15.4 standard does not define a protocol but only a standard for the physical and data link layers, so specific protocols as Zigbee have been developed on top of it to provide all the necessary functionalities and features for enabling communication in IoT systems.

3.1.1. Physical Layer (PHY)

The Physical Layer (PHY) of the IEEE 802.15.4 standard defines the physical characteristics and parameters of the wireless communication, such as the frequency bands where the communication can take place. It defines three different frequency bands that can be used for communication:

- 868 MHz Band: This band is used in Europe and other regions, and it provides a data rate of up to 20 kbps. It has one channel available for communication.

- 915 MHz Band: This band is used in North America and other regions, and it provides a data rate of up to 40 kbps. It has ten channels available for communication.

- 2.4 GHz Band: This band is used worldwide, and it provides a data rate of up to 250 kbps. It has sixteen channels available for communication.

The services provided by the PHY layer are divided into two categories:

- Data Services: These services are used for transmitting and receiving the PPDU between devices in the PAN.

- Management Services: These services are used for managing the communication and the network, such as channel scanning (to find the best channel for communication), link quality indication (to measure the quality of the communication link between devices) and CCA (Clear Channel Assessment, to check if the channel is idle before transmitting).

The content that is in fact transmitted over the medium is the *PHY Protocol Data Unit (PPDU)*, which consists of the *PHY Service Data Unit (PSDU)* encapsulated with the header. The 6 bytes Header consists of:

- Synchronization Header (SHR), 5 bytes:

  This header is used for synchronization between the transmitter and the receiver, and it consists of a preamble (4 bytes) and a start of frame delimiter (1 byte).

- PHY Header (PHR), 1 byte:

  The first 7 bits of this header are used to indicate the length of the PSDU, while the last bit is reserved for future use. Therefore, the maximum length of the PSDU is $2^7 - 1 = 127$ bytes.

3.1.2. Data Link Layer (MAC)

The standard IEEE 802.15.4 only defines the MAC sublayer of the Data Link Layer, which is responsible for managing access to the medium and ensuring reliable communication between devices in the PAN. The services provided by the MAC layer are divided into two categories:

- Data Services: These services are used for transmitting and receiving the MPDU between devices in the PAN across the PHY layer.

- Management Services: These services are used for managing the communication and the network, such as the beacon or GTS management. It also has hooks[1] for implementing security services.

[1]It has hooks (and not specific services) because the security services are eventually broken, so the standard would end up being unsecure nd obsolete. However, providing hooks allows for selecting the optimal security services and replacing them when they are broken without the need to change the standard itself.

The content that is in fact transmitted over the medium is the *MAC Protocol Data Unit (MPDU)*, which consists of the *MAC Service Data Unit (MSDU)* encapsulated with the header and the footer. The footer is just a 2 bytes CRC (Cyclic Redundancy Check) for error detection, while the header consists of:

- Frame Control (FC), 2 bytes:

  This field is used to indicate the type of frame being transmitted, such as a data frame, a beacon frame, or an acknowledgment frame.

- Sequence Number, 1 byte:

  This field is used to identify the frames being transmitted, which can be useful for ensuring reliable communication and detecting duplicate frames.

- Addressing Fields, 0 to 20 bytes:

  These fields are used to indicate the source and destination addresses of the frame being transmitted, and the PAN ID (2 bytes) of each device.

Regarding the MAC adresses, there are two types of addresses that can be used in the IEEE 802.15.4 standard:

- Short Address: This is a 16-bit address that is manually assigned to a device by the network operator. It is used for communication within the PAN, and it is not globally unique.

  These are called Short Addresses (SADDR).

- Extended Address: This is a 64-bit address that is asigned to a device by the manufacturer, and it is globally unique. It is used for communication between devices in different PANs, and it can also be used for communication with devices on the Internet.

  These are called Extended Unique Identifier (EUI-64) addresses.

  Specially important is the second least significant bit of the first byte of the EUI-64 address, which is called the universal/local (U/L) bit.

This standard also defines several types of variants for the MAC layer, which esentially use different variants of controlling the access to the medium.

CSMA/CA and variants

The Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) is a protocol for controlling access to the medium in a shared communication channel. It is based on the principle of carrier sensing, where devices listen to the channel before transmitting to ensure that it is not already in use by another device. Its main idea is described in Figure 3.1, where a backoff time is waited because:

- If multiple devices want to transmit at the same time and no backoff time is used, they would all start transmitting simultaneously when they sense that the channel is idle, which would lead to collisions and failed transmissions.

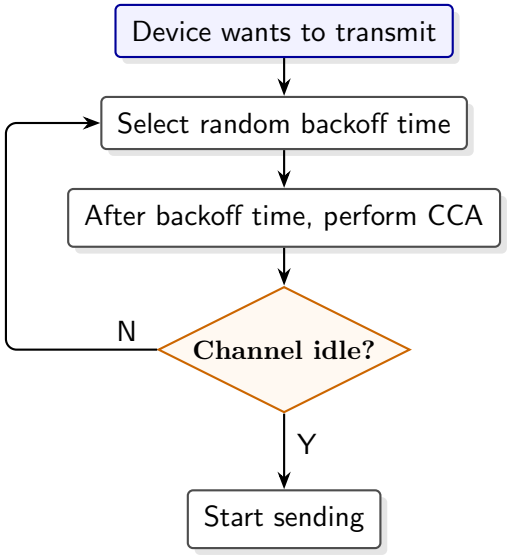

Figura 3.1: CSMA/CA protocol.

- It saves energy, as the device can go to sleep during the backoff time and wake up just before attempting to transmit, which can be more efficient than continuously sensing the channel.
- It allows for better performance, as it allows for the ackoledgment of successful transmissions and the retransmission of failed transmissions, which can improve the overall reliability and efficiency of the communication.

However, this main protocol has different specific variants that are used in the different variants of the MAC layer defined in the IEEE 802.15.4 standard.

- Slotted CSMA/CA:

  This variant is used when the frame is divided into slots. The main idea of this protocol is described in Figure 3.2. Some important abbreviations used there are:

  - *BE*: backoff exponent, which is used to calculate the possible number of backoff periods[2] that the device can choose for waiting.
  - *NB*: number of backoffs, which is the number of times the device has attempted to access the channel and has been unsuccessful.
  - *CW*: contention window, which is the number of consecutive clear channel assessments (CCAs) that the device needs to perform before it can start transmitting. It is used to ensure that the device has a good chance of successfully accessing the channel before it starts transmitting.

- Unslotted CSMA/CA:

  This variant is used when the time is not divided into slots. The main idea of this protocol is described in Figure 3.3. Here, the CW is not used as there are no slots, so the device can attempt to transmit as soon as it senses that the channel is idle, without waiting for the next slot.

[2] A backoff period is the basic unit of time when using this protocol.

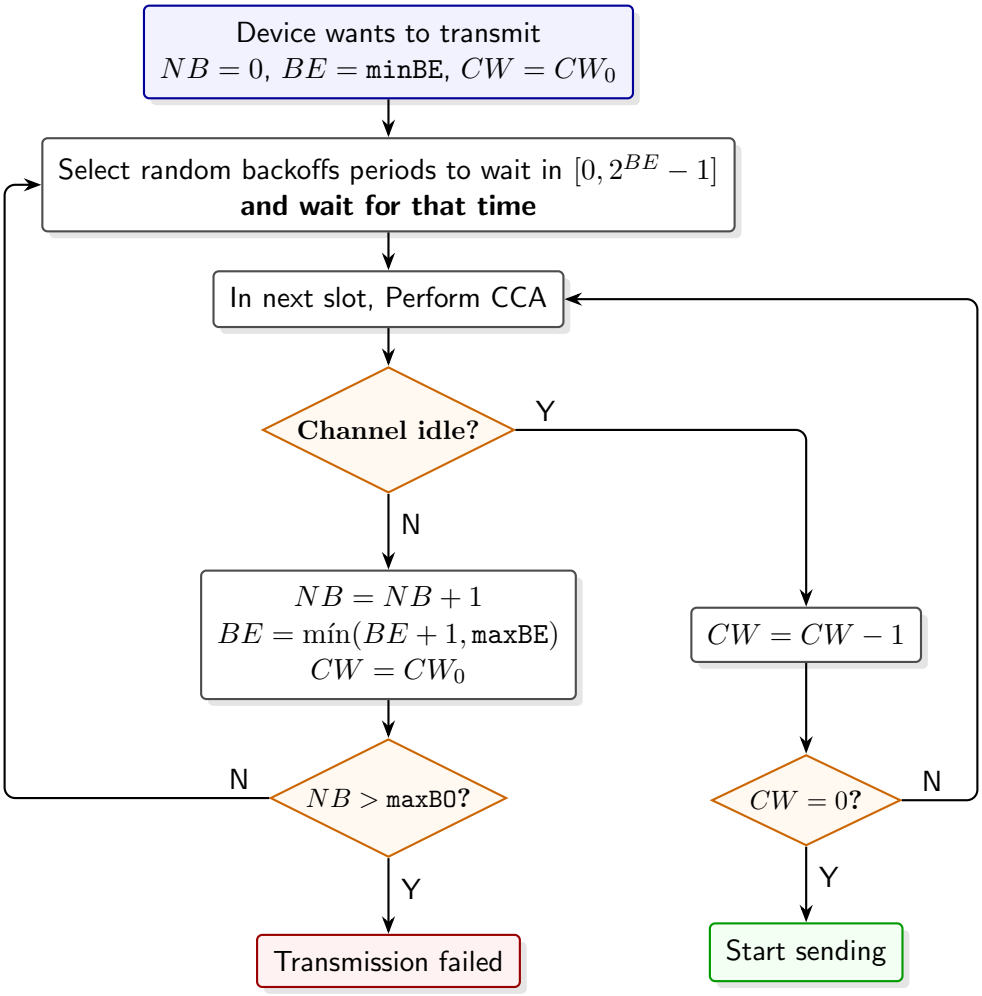

Figura 3.2: Slotted CSMA/CA protocol.

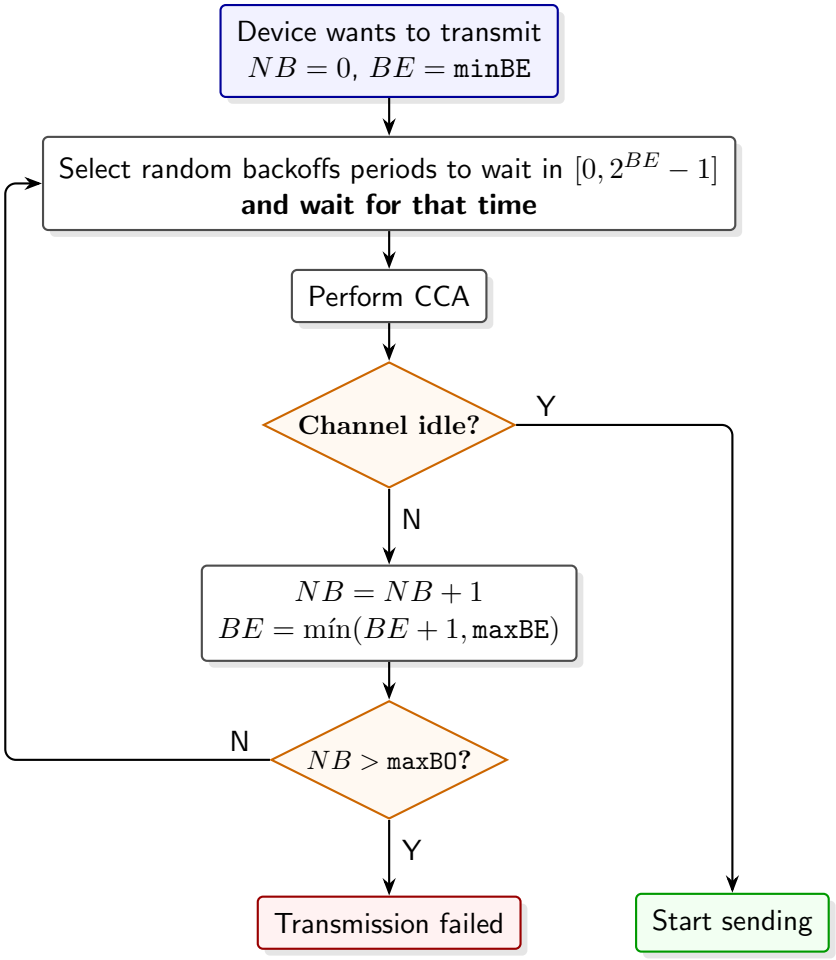

Figura 3.3: Unslotted CSMA/CA protocol.

- Time Division Multiple Access (TDMA):

  This is another variant used for controlling access to the medium when more than one device needs to transmit at the same time. In this protocol, the devices are assigned the whole frequency band for a specific period of time, which is called a time slot.

- Frequency Division Multiple Access (FDMA):

  This is another variant used for controlling access to the medium when more than one device needs to transmit at the same time. In this protocol, the devices are assigned different frequency bands for communication, which allows them to transmit simultaneously without interference.

- Time-Frequency Division Multiple Access (TFDMA):

  This last variant is a combination of TDMA and FDMA, where the devices are assigned only a portion of the frequency band for a specific period of time, which allows for more efficient use of the available resources and can improve the performance of the communication.

Once the different forms of controlling access to the medium have been described, the specific variants of the MAC layer defined in the IEEE 802.15.4 standard can be described, which basically use different variants of controlling access to the medium.

Beacon-enabled Networks

Beacon-enabled networks are a variant of the MAC layer defined in the IEEE 802.15.4 standard, where time is divided into superframes. Each superframe starts with a *beacon frame* that is transmitted by the PAN Coordinator to synchronize the devices in the PAN and to provide information about the network, such as the PAN ID, the channel, and the superframe structure. It is divided in:

- Active Period: During this period of time, the devices in the PAN can communicate with each other. The active period is divided into 16 slots, and they are split into two parts (configurable by the PAN Coordinator):
  - *Contention Access Period (CAP)*: During these time slots, the devices in the PAN compete for access to the medium using the slotted CSMA/CA protocol to transmit their data. It should be noted than during the CAP, devices will only start transmitting if they have enough time slots (CW) to complete the transmission of their data.
  - *Contention Free Period (CFP)*: These time slots are called *Guaranteed Time Slots (GTS)*, and there are up to 7 GTSs available in each superframe. Devices in the PAN can request GTSs from the PAN Coordinator, which assigns them to the devices that request them. During these time slots, the devices that have been assigned GTSs can transmit their data without competing for access to the medium.
- Inactive Period: During this period of time (configurable by the PAN Coordinator), no communication takes place, and the devices in the PAN (including the PAN Coordinator) can go to sleep to save energy.

In this variant, there is also an Acknowledgment mechanism. In the header of a data frame sent, there is a specific bit that indicates whether the sender of the frame wants to receive an acknowledgment from the receiver (except from beacon and acknowledgment frames, which do not allow for acknowledgment). If this bit is set:

- If the receiver successfully receives the frame, it sends an acknowledgment frame back to the sender to confirm that the frame has been received correctly.
- If the sender does not receive an acknowledgment frame within a certain period of time, it assumes that the transmission has failed and attempts to retransmit the frame. There is a maximum number of retransmissions that the sender can attempt before giving up and considering the transmission as failed.

Networks without Beacons

This is another variant of the MAC layer defined in the IEEE 802.15.4 standard, where there are no beacon frames and no superframes. In this variant, the devices in the PAN continuously compete for access to the medium using the unslotted CSMA/CA protocol to transmit their data.

Time Slotted Channel Hopping (TSCH) MAC Layer

This new modern variant of the MAC layer defined in the IEEE 802.15.4 standard is designed to provide high reliability and low latency communication for IoT applications. The PAN coordinator divides time into slots, which are grouped into *slotframes*. Each slotframe consists of a fixed number of time slots, and the communication between devices in the PAN is organized around these slotframes. In addition, different frequency channels are used for communication, so a TFDMA approach is used for controlling access to the medium. Each time slot for each frequency channel is called a *cell*, and there are three types of cells:

- Shared Cells: These cells are used for communication between devices in the PAN, and they are shared among all the devices in the PAN. They use CSMA/CA for controlling access to the medium.
- Dedicated Cells: These cells are assigned to specific devices in the PAN, and they are used for communication between those devices. They are assigned using a pseudo-random algorithm[3] (it is not totally random because it needs to be deterministic to ensure that the devices know which cells they can use for communication), and they do not use CSMA/CA for controlling access to the medium, as the devices that have been assigned those cells can transmit their data without competing for access to the medium.
- Advertisement Cells: These cells are used for transmitting frames from the PAN Coordinator to the devices in the PAN.

[3]IEEE 802.15.4 does not specify this. It is delegated to higher layer protocols.

It should be noted that this variant also uses an acknowledgment mechanism similar to the one used in the beacon-enabled networks, and both the data and the acknowledgment frames are supposed to be transmitted within the same cell.

For each slotframe, and given that there are multiple channels (frequencies) available for communication, the PAN Coordinator assigns a specific channel to each time slot in the slotframe, which is called *channel hopping*.

3.2. NB-IoT

Even though the IEEE 802.15.4 standard and its variants are widely used in IoT applications, they have a limited range, which can be a problem for some IoT applications that require long-range communication. To address this issue, the Narrowband Internet of Things (NB-IoT) standard was developed to provide a low-power, wide-area (LPWA) communication solution for IoT applications. It is based on the Long-Term Evolution (LTE) standard, which is widely used for cellular communication.

3.2.1. LTE and NB-IoT

LTE (Long-Term Evolution) is a standard for wireless communication that is widely used for cellular communication over a licensed spectrum. It divides the whole area in smaller regions called *cells*, and each cell is served by a base station that provides wireless communication to the devices in that cell. It has a layered architecture that consists of three main layers:

- Host-to-Network Layer: This is the part of the LTE network that provides wireless communication to the devices in the cells. Each cell is served by a base station (called `eNB`, evolved Node B) that provides wireless communication to the devices (called `UE`, User Equipment) in that cell. The cellular network is responsible for managing the communication between the devices and the base stations.

- Core Network Layer: This is the part of the LTE network that provides the necessary infrastructure and services for enabling communication between the different cells and the Internet. They provide most of the functionalities and features of the LTE network, such as routing, mobility management, security, and quality of service (QoS). In this layer, multiple servers are used:

  - *Mobility Management Entity (MME)*: This server is responsible for managing the mobility of the devices in the LTE network, such as tracking their location and handling handovers between cells when they move.
  - *Home Subscriber Server (HSS)*: This server is responsible for storing the subscriber information and providing authentication and authorization services for the devices in the LTE network. Since LTE works in a licensed spectrum, only authorized devices can connect to the network, and the HSS is responsible for ensuring that.

    - *Serving Gateway (SGW)*: This server is responsible for routing the data packets between the devices in the LTE network and the Internet, and it also provides some security and QoS features.
    - *Packet Data Network Gateway (PGW)*: This server is responsible for providing access to external networks, such as the Internet, and it also provides some security and QoS features.

- Application Layer: These are the networks that are connected to the LTE network, such as the Internet or other networks. They provide access to external resources and services for the devices in the LTE network. Here, the usual TCP/IP protocols are used for communication.

Once installed, a cellular network enables connection to the Internet everywhere in the area covered by the network (which is usually really large). However, setting up a cellular network can be really expensive, and it may not be feasible for some IoT applications that require low-cost communication solutions. To address this issue, the NB-IoT standard was developed to provide a low-cost, low-power, wide-area communication solution for IoT applications that can be deployed in existing LTE networks without the need for changes to the infrastructure.

3.2.2. NB-IoT Channels

NB-IoT can be deployed in three different ways, which use different channels for communication:

- Standalone Deployment: This deployment is used when there is no other system available and therefore GSM (which is an older cellular communication standard) spectrum is used for communication. In this deployment, NB-IoT reserves a specific portion of the GSM spectrum for communication (a single 200 kHz channel), and it uses that channel for communication between the devices and the base stations.

  As an advantage, this deployment does not interfere with the existing LTE network, as it uses a different spectrum for communication. However, GSM is older and less efficient than LTE, so this deployment may not provide the best performance for NB-IoT applications.

- Guard Band Deployment: This deployment is used when there is an existing LTE network, and it uses the guard band (which is a portion of the spectrum that is reserved for preventing interference between different channels) for communication. This band is not used for communication in the LTE network, so it can be used for NB-IoT communication without interfering with the existing LTE network. A channel is then reserved in the guard band for NB-IoT communication. It should however be used with caution, as the guard band is not designed for communication and therefore may have some limitations in terms of performance and reliability.

- In-Band Deployment: This deployment is used when there is an existing LTE network, and it uses a portion of the LTE spectrum for communication. In

this deployment, a specific portion of the LTE spectrum (a single channel) is used for NB-IoT communication, and it is time-shared with the existing LTE network. Collisions between NB-IoT and LTE communication need to be controlled to ensure that both types of communication can coexist without interference.

However, these physical channels are not directly used, but they are abstracted by logical channels that provide specific functionalities and features for enabling communication in NB-IoT applications. NB-IoT uses specific logic channels, and each of them is divided in two different types of channels:

- Signaling Channels: These channels are used for transmitting control information and signaling messages between the devices and the base stations. They are used for tasks such as device registration, authentication, and mobility management.
- Data Channels: These channels are used for transmitting user data between the devices and the base stations.

Each of these channels in fact uses several physical channels for communication, and they have to be mapped. There are two different mappings that are applied:

- Frequency Multiplexing: Two different physical channels are used, one for uplink communication (from the device to the base station) and another for downlink communication (from the base station to the device).
- Time Multiplexing: In the same physical channel, different time slots are used for different logical channels, so the devices and the base stations need to be synchronized to ensure that they are using the correct time slots for communication.

3.2.3. Cell Controlling in NB-IoT

NB-IoT has a well-known common model: messages will be infrequent and small. This allows for no handover. While you are being served by a base station, you can perfetly communicate with it, but if you move to another cell, the communication will be lost and you will need to join to the new cell. In order to join a cell, the following process is followed:

1. The UE listens in a broadcast channel for a specific message from the base station containing the information about the cell.
2. After receiving that message, the UE checks if the cell is joinable (as there are test cells, for example, that are not joinable).
3. If the cell is joinable, in the information received there is a threshold for the signal strength, so the UE checks if the signal strength is above that threshold.
4. From all the joinable cells with a signal strength above the threshold, the UE selects the one with the best signal strength and tries to join it.

Switching between cells is also supported but, as previously mentioned, there is no handover, so the communication will be lost during the switching process. The switching process can be performed when, due to mobility or other factors, the signal strength of the current cell drops below a certain threshold, and a new cell with a better signal strength is available. In that case, the UE can switch to the new cell by following the same process as when joining a cell for the first time.

3.2.4. UE Modes in NB-IoT

NB-IoT defines four different modes for the UE, which are designed to optimize the power consumption of the devices in the NB-IoT network.

Active Mode

An UE only enters this mode when it needs to transmit or receive data, and it is the most power-consuming mode. In this mode, the UE is actively communicating with the base station, and therefore has reserved resources for communication. There are two different types of transmission in this mode:

- Uplink Transmission: In this type of transmission, the UE transmits data to the base station. There are two different variants:
  - *Via Data Channels*: This is a connection-oriented transmission. The UE uses a signal channel to request resources for uplink transmission, and the base station informs the UE about the timeslots and the data channels that it can use for the uplink transmission. The UE can then transmit its data to the base station using those resources when needed. After the transmission is completed, the UE can release the resources that it has been using for the uplink transmission and close the connection.

    This is useful for large transmissions, but introduces a lot of overhead for small transmissions. An small optimization is used by letting the UE ask the base station to resume the connection that it had previously established, so the UE can use the same resources that it had been using before without the need to go through the whole process of requesting resources and establishing a new connection. However, the overhead is still significant for small transmissions, so there is another variant for small transmissions.
  - *Via Signal Channels*: This is a connectionless transmission. In the first step, when the UE asks the base station for resources for uplink transmission, the actual data that the UE wants to transmit is included in the request (called piggybacking), so there is no need to establish a connection and reserve resources for the transmission. The base station can then transmit the data to the core network immediately after receiving the request from the UE, without the need to wait for the UE to transmit its data using the reserved resources. This is useful for small transmissions, as it allows for a more efficient use of the resources and reduces the overhead associated with establishing a connection and reserving resources for the transmission.

- Downlink Transmission: In this type of transmission, the base station transmits data to the UE. The base station pages the UE (it can page several UE in the same message) to notify it that there is data waiting for it to be transmitted. Once the UE receives the paging message, there are also the two same variants for the downlink transmission:
  - *Via Data Channels*: Connection-oriented transmission where the UE reserves a specific timeslot and data channel for the downlink transmission.
  - *Via Signal Channels*: Connectionless transmission where the UE tells the base station that it is ready to receive the data, and that the data should be included (piggybacked) in the message sent over the signal channel, so there is no need to reserve resources for the downlink transmission.

Regarding the downlink transmission, paging is used to notify the UE that there is data waiting for it to be transmitted. However, once the gateways receive the data for the UE, it is not yet known which base station is serving the UE, i.e., which base station should send the paging message to the UE. To solve this issue, tracking is used. Two extreme scenarios could be considered to better understand how it is done.

- *Only one cell is paged*: The MME keeps track of the cell that is serving the UE at every moment, so when there is data waiting for the UE, only its specific cell is paged. This would reduce the overhead of paging (as only one cell is paged), but it would imply a lot of overhead each time the UE switches between cells. This is also a problem from the privacy point of view, as the MME would have to keep track of the cell that is serving the UE at every moment, which could be used to track the location of the UE.
- *All the cells are paged*: The MME does not keep track of the cell that is serving the UE, so when there is data waiting for the UE, all the cells are paged. This would reduce the overhead of tracking (as there is no need to keep track of the cell that is serving the UE), but it would imply a lot of overhead for paging (as all the cells are paged).

As the reader may have already guessed, the actual solution is a trade-off between these two extreme scenarios. The concept of *Tracking Area* is used, which is just a group of cells with the same tracking area identifier (TAI). The MME keeps track of the tracking area that is serving the UE, so when there is data waiting for the UE, only the cells in that tracking area are paged. When the UE needs to inform the MME of the cell that is serving it, it sends a Tracking Area Update (TAU) message to the MME, which updates in its internal records the tracking area that is serving the UE.

- When a UE joins the network, it sends a TAU message to the MME to inform it of the tracking area that is serving it.
- When a UE switches between cells, it checks if the new cell belongs to the same tracking area as the previous cell. If it does, there is no need to send a TAU message to the MME, as the tracking area that is serving the UE has not changed. If it does not, the UE sends a TAU message to the MME to inform it of the new tracking area that is serving it.

- Every certain period of time, the UE sends a TAU message to the MME to inform it that it is still connected to the network. If the MME does not receive a TAU message from the UE within a certain period of time, it assumes that the UE has disconnected from the network and it deletes its internal record accordingly.

Idle Mode

This mode is used when the UE does not need to transmit or receive data, but it still needs to be registered in the network and able to receive paging messages from the base station. No resources are reserved for communication in this mode, but:

- The UE still informs the base station about its location, so the base station can send paging messages to the UE when needed.
- The UE still monitors for paging messages from the base station, so it can receive them when they are sent.
- The SGW retains the incoming data for the UE in buffers until it receives a paging message from the base station, which indicates that there is data waiting for the UE to be transmitted or received.

Even though a UE can set itself to idle mode, the base station can also set the UE to idle mode if it detects that the UE has not been transmitting or receiving data for a certain period of time.

Extended Discontinuous Reception (eDRX) Mode

One thing that should be taken into account is that paging is not done continuously, but it is done in specific time intervals are the start of each hyperframe (which is a group of slotframes). This means that the UE needs to be listening for paging messages only at the start of each hyperframe, and it can go to sleep during the rest of the time to save energy.

The eDRX mode is used when the UE needs to save energy by sleeping for longer periods of time, but it still needs to be able to receive paging messages from the base station. In this mode, the UE can configure the number of hyperframes that it will sleep for, so it will only listen for paging messages at the start of each hyperframe after sleeping for that number of hyperframes. This allows the UE to save energy by sleeping for longer periods of time, while still being able to receive paging messages from the base station when needed. During all that time, the SGW retains the incoming data for the UE in buffers.

Power Saving Mode (PSM)

This mode is used when the UE needs to save even more energy by sleeping for an unlimited period of time. During this time, the base station will not page the UE, and the SGW will retain the incoming data for the UE in buffers. When the UE awakes from this mode to send data, it will notify the base station that it is awake and wait for the base station to send the buffered data to it (if there is any data waiting for it to be transmitted or received).

3.3. 6LowPAN

During this section, we will cover how to actually connect the devices to the internet, which is the main goal of IoT. In order to do so, IP (Internet Protocol) is used, which is the main protocol for communication in the Internet. In order to connect to internet, there are two different approach, depending on the protocols used in the lower layers.

3.3.1. IP with NB-IoT

When a UE uses NB-IoT for communication, given that NB-IoT is based on LTE (which is fully IP-based), LTE provides the direct support needed for connecting the UE to the Internet using IP. There are two options:

- If the UE is powerful enough to directly speak IP:

  The PGW can asign an IP address to the UE, and the UE can use that IP address to communicate with other devices in the Internet using IP.

- If the UE is not powerful enough to directly speak IP:

  In this case, the UE uses an specific protocol for communication with the SGW, which forwards the data to the PGW, which is responsible for translating that specific protocol to IP and forwarding the data to the Internet. Therefore, the PGW acts as a gateway between the UE and the Internet, creating the IP packets.

  It should however be noticed that there is also an IP address assigned to the UE, but it is not used for communication between the UE and the base station, but it is only used for communication between the PGW and the Internet. The UE does not even know that it has an IP address.

3.3.2. IP with IEEE 802.15.4: 6LowPAN

When a device uses IEEE 802.15.4 for communication, given that IEEE 802.15.4 is not IP-based, there is no direct support for connecting the device to the Internet using IP. A first approach to solve this issue is to use the PAN Coordinator as a gateway between the devices in the PAN and the Internet. However, in this section we will cover how to directly connect the devices in the PAN to the Internet using IP.

A first approach would be to directly use IP, but there are some issues (which will be covered) that makes it necessary to use a specific protocol for connecting the devices in the PAN to the Internet using IP, which is called 6LowPAN (IPv6 over Low-Power Wireless Personal Area Networks). This protocol is just an adaptation layer to let IP and IEEE 802.15.4 work together. The IP SDU is encapsulated using a header stack, where the following headers appear *in the following order*:

1. Mesh Addressing Header: Used when the peer-to-peer communication is needed between the devices in the PAN, as it allows for multi-hop communication between the devices in the PAN.

2. Broadcast Header: Used when a broadcast communication is needed between the devices in the PAN.

3. Fragmentation Header: Used when the IP SDU is too large to be transmitted in a single IEEE 802.15.4 frame.

4. Dispatch Header (1 byte): This header is used to indicate the type of the following headers in the header stack (regarding compression).

It should be noted that, if any of those headers is not needed, it can be omitted to save resources. In order to better understand how does 6LowPAN work, we will cover the main issues that arise when trying to directly use IP over IEEE 802.15.4, and how 6LowPAN solves those issues.

Limited IP Adresses Configuration

The IoT Things have almost no user interaction, so they need to be able to automatically configure their IP address when they join the network. IPv6 provides a general network-independent mechanism for automatic IP address configuration, but it does not take into account the specifications of IEEE 802.15.4.

The solution that 6LowPAN provides for this issue is **Stateless Address Autoconfiguration (SLAAC)**, which is a mechanism that allows the devices in the PAN to automatically configure their IP address when they join the network, without the need for a DHCP server or any other external configuration mechanism (that is why it is called stateless). The addresses assigned are *Local Link (LL) IPv6 addresses*, which are only valid within the local network (the PAN) and cannot be used for communication with devices outside the PAN. If communication with devices outside the PAN is needed, then the router in the PAN (which is usually the PAN Coordinator) will perform a Network Address Translation (NAT) to translate the LL IPv6 addresses to global IPv6 addresses that can be used for communication with devices outside the PAN.

The 16-byte IPv6 addresses are created taking into account the IEEE 802.15.4 addresses, so it depends on the type of address used:

- 64-bit Extended Unique Identifier (EUI-64) Address:

  The IPv6 address has two different parts: the IPv6 Interface Identifier (IID), which is the 8 least significant bytes of the IPv6 address, and the IPv6 Network Prefix, which is the 8 most significant bytes of the IPv6 address.

  - The Network Prefix is a fixed value that is defined by the 6LowPAN standard (given that it is only used for communication within the PAN, there is no need for a global unique prefix). It is set to `FE80::`[4], which is the prefix for link-local addresses in IPv6.
  - The IID is created using the EUI-64 address of the device, but just flipping the U/L bit.

[4] In IPv6, hexadecimal notation is usually used. The usual convention is to split the address in 2-bytes groups, using ":" to differenciate them. Two consecutive colon "::" is used to filled with as many zeros as needed.

- 16-bit Short Address:

  The Network Prefix is the same as in the previous case, but the IID is created using the 16-bit short address of the device. The SADDR is "converted" to the following 64-bit address: `<PAN ID>:00FF:FE00:<SADDR>`, where `<PAN ID>` is the PAN ID of the device, and `<SADDR>` is the 16-bit short address of the device. The IID is then created using that 64-bit address, but just setting the U/L bit to 0.

This mechanism allows the devices in the PAN to automatically configure their IP address when they join the network, without the need for a DHCP server or any other external configuration mechanism, and it also allows for communication between the devices in the PAN using their IPv6 addresses.

IPv6 Minimum MTU requirement

IPv6 requires that the minimum MTU for communication is 1280 bytes, which is much larger than the maximum frame size of IEEE 802.15.4 (127 bytes). The IPv6 protocol states that, for communication over links with a smaller MTU, link-specific fragmentation and reassembly mechanisms need to be implemented at a lower layer, which is what 6LowPAN does. When fragmenting, the fragment header is added to the header stack, and it differs from the first fragment to the rest of fragments.

- The fragment header for the first fragment contains:
  - *Fragment Type* (5 bits): `11000` for the first fragment.
  - *Datagram Length* (11 bits): This field indicates the total length (in bytes) of the original IPv6 PDU that the 6LowPAN header stack is encapsulating, including the IPv6 header and the payload (after the IP fragmenting). It is used, for instance, to let the receiver know how much buffer space it needs to allocate for reassembling the fragments.

    It should be noted that the maximum value for this field is $2^{11} - 1 = 2047$ bytes, which is larger than the minimum MTU required by IPv6. However, the MTU for communication over IEEE 802.15.4 using 6LowPAN is 1280 bytes (the minimum MTU required by IPv6), which is the minimum MTU required by IPv6.
  - *Datagram Tag* (16 bits): This field is used to identify the fragments that belong to the same original IPv6 PDU. All the fragments that belong to the same original IPv6 PDU will have the same value in this field, so the receiver can use it to reassemble the fragments correctly.
- The fragment header for the rest of fragments contains:
  - *Fragment Type* (5 bits): `11100` for the rest of fragments.
  - *Datagram Length* (11 bits): This field has the same value as in the first fragment, and should be repeated in all the fragments because the second fragment may arrive before the first fragment, so the receiver needs to know the total length of the original IPv6 PDU to be able to reassemble the fragments correctly.

- *Datagram Tag* (16 bits): This field has the same value as in the first fragment.
- *Datagram Offset* (8 bits): This field indicates the offset (in multiples of 8 bytes) of the data in the current fragment with respect to the start of the original IPv6 PDU. It is used by the receiver to reassemble the fragments in the correct order.

  It should be noted that the maximum value for this field is $2^8 - 1 = 255$, which means that the maximum size of each fragment is $255 \cdot 8 = 2040$ bytes. Given that the maximum length of the original IPv6 PDU is 1280 bytes, that offset is enough to ensure that the fragments can be reassembled correctly.

  In addition, it should be noted that the capacity available with IEEE 802.15.4 for the payload is sometimes not fully used, as the offset of the next fragment needs to be a multiple of 8 bytes.

Regarding defragmentation, a fragment could be lost during transmission, so the receiver needs to implement a timeout mechanism to avoid waiting indefinitely for the missing fragment. If the receiver does not receive all the fragments of an original IPv6 PDU within a certain period of time, it discards all the fragments that it has received for that original IPv6 PDU (as it is considered as a failed transmission). Another higher layer (as TCP) can then request the retransmission of the original IPv6 PDU if needed.

Header Overhead

Even though up to this point IP and IEEE 802.15.4 have been made to work together, this would still be very inefficient. Out of the 127 bytes of the maximum PSDU size of IEEE 802.15.4:

- Up to 25 bytes are used for the MAC header and the MAC footer, which leaves 102 bytes for the payload.
- Up to 21 bytes are used for an additional encryption protocol, which leaves 81 bytes for the payload.
- If 6LowPAN is used, even more bytes are used for the 6LowPAN header stack, which leaves even less bytes for the payload.
- 40 bytes are used for the IPv6 header, which leaves 41 bytes for the payload.
- 8 bytes are used for the UDP header, which leaves 33 bytes for the payload.

Having a payload of only 33 bytes is really inefficient, so 6LowPAN provides a mechanism for compressing the IPv6 header to reduce the overhead of transmitting IP packets over IEEE 802.15.4. Two different approeaches for compression are used: HC1 and HC2, or IPHC and NHC. We will cover in detail the HC1 and HC2 mechanisms, as they are the most widely used ones:

- IPv6 Header Compression (HC1): This is a simple compression mechanism that is used for compressing the IPv6 header. In the Dispatch Header, `0x42` is used to indicate that the IPv6 header is compressed using HC1. After that, the HC1 encoding indicates which fields of the IPv6 header are compressed and which fields are not compressed. The HC1 encoding fields are:

  - 2 bits for each of the address fields (source and destination).

    Each of the address are split in two parts: the network prefix (the 8 most significant bytes) and the interface identifier (the 8 least significant bytes).

    - The first bit indicates if the network prefix is assumed to be the LL IPv6 prefix (which is `FE80::`) or not. If assumed, a 1 is set in that bit and the network prefix is not included in the compressed header. If not assumed, a 0 is set in that bit and the network prefix is included in the uncompressed fields.
    - The second bit indicates if the interface identifier is assumed to be derived from the IEEE 802.15.4 address of the device or not. If assumed, a 1 is set in that bit and the interface identifier is not included in the compressed header. If not assumed, a 0 is set in that bit, and the interface identifier is included in the uncompressed fields.

    It should be noted that, when the address is not assumed (which is common when the communication needs to be done with devices outside the PAN), the whole address is included in the uncompressed fields, so the compression is not really effective. IPHC and NHC tend to solve this problem.

  - 1 bit for the Traffic Class and Flow Label fields:

    If both fields are assumed to be 0 (which is the most common case), a 1 is set in that bit. If they are not assumed, a 0 is set in that bit, and both fields are included in the uncompressed fields.

  - 2 bits for the Next Header field:

    00 means that the Next Header field is not compressed, and it is included in the uncompressed fields. 01 stands for UDP, 10 stands for ICMP, and 11 stands for TCP, so in those cases the Next Header field is compressed by just indicating the protocol used in that field.

    This is sometimes a problem, because there are more types of headers (as the fragment header, if fragmentation in IP has happened), that are not considered.

  - 1 bit for deciding if HC2 encoding is used for compressing the next header:

    If the Next Header field is compressed using the previous 2 bits, that header can also be compressed using HC2. If it is used, a 1 is set in that bit, and the next header is compressed using HC2 encoding. If it is not used, a 0 is set in that bit.

    If the Next Header field is not compressed using the previous 2 bits, then HC2 can't be used for compressing the next header, so a 0 is set in that bit.

Some specific fields of the IPv6 header were not mentioned in the HC1 encoding:

- Version: This field is always set to 6 for IPv6, so it is not needed to be included in the compressed header.
- Payload Length: This field can be calculated from the length of the IEEE 802.15.4 frame.
- Hop Limit: This field can't be assumed to have a specific value, so it is always sent afterwards uncompressed.

The order of the uncompressed fields (if they are present) is: hop limit, source address, dest address, traffic class, flow label and next header[5].

It should be noted that, in a worst case, no compression but the opposite would happen. In that case, we can just send the uncompressed IPv6 header, but no HC2 compression would be possible.

- Next Header Compression (HC2): This is another compression mechanism that is used for compressing the next header when it is a UDP, TCP or ICMP header, the field "Next Header" has been compressed in HC1, and the HC2 field in HC1 was activated.

  The HC2 encoding for a UDP[6] Header is as follows:

  - 1 bit for each of the ports (source and destination):

    Each port can only be compressed if it is in the range [`F0B0`, `F0BF`]. If it is in that range and it is compressed, a 1 is set in that bit, and the port is compressed by just including the least significant 4 bits of the port number in the uncompressed fields. If it is not in that range or it is not compressed, a 0 is set in that bit, and the 16 bits of the port number are included in the uncompressed fields.
  - 1 bit for the length field:

    If the length field is derived from the length of the IPv6 payload, a 1 is set in that bit, and the length field is not included in the uncompressed fields. If it is not, a 0 is set in that bit, and the length field is included in the uncompressed fields.
  - 5 bits reserved for future use, not used.

  Some specific fields of the UDP header were not mentioned in the HC2 encoding:

  - Checksum: This field can't be assumed to have a specific value, so it is always sent afterwards uncompressed.

  The order of the uncompressed fields (if they are present) of the UDP header is: source port, destination port, length and checksum.

[5] After the hop limit (which is always present), they follow the same order as in the HC1 encoding.

[6] TCP and ICMP follow similar, but more complex, schemes.

If HC1 and HC2 compression is used, the order is quite unexpected. After the Dispatch Header, the HC1 and, if used, the HC2 encoding are included. After them, the HC1 uncompressed fields are included and, if HC2 is used, the HC2 uncompressed fields are finally included.

However, also the IPHC (IP Header Compression) and NHC (Next Header Compression) mechanisms can be used for compression. Some of their benefits are:

- They include mechanisms for compressing both routed addresses and broadcast addresses.

  They do not compress packets on their own, but also considering that they may be the same as in the previous packet. If the previous packet is not available at destination (because the order is not guaranteed), they can be whether buffered at destination or simply discarded (as IP is not reliable).

- The hop limit field can be also compressed.

- Higher layer headers can be better compressed without mixing the compression of the IPv6 header and the compression of the next header, as it happens with HC1 and HC2.

3.4. MQTT

During this last section, we will cover the application layer protocols. Apart from other protocols as HTTP, MQTT (Message Queuing Telemetry Transport) is one of the most widely used application layer protocols in IoT applications. It is a lightweight publish-subscribe messaging protocol which uses TCP (even though it introduces some overhead). It follows a logical *star topology*, where there is a central *broker* that is responsible for receiving messages from the publishers and forwarding them to the subscribers.

Observación. This local star topology should not be confused with the topology of the underlying network, which can be any topology (as a mesh topology). The publishers and subscribers can be connected to the broker through any type of network topology, as long as they can reach the broker to send and receive messages.

This protocol uses a *publish-subscribe* communication model, which is different from the traditional *request-response* communication model used in protocols like HTTP. In the publish-subscribe model, there are two steps for communication.

- Subscribe: When a client is interested in receiving messages about a specific topic, it subscribes to that topic by sending a subscribe message[7] to the broker. The broker then adds the client to the list of subscribers for that topic, so it will receive any messages published to that topic in the future.

  Unsubscribe is also supported, so if a client is no longer interested in receiving messages about a specific topic, it can send an unsubscribe message to the broker to remove itself from the list of subscribers for that topic.

[7] A subscribe message could contain several topics at the same time.

- Publish: When a client wants to send a message about a specific topic, it publishes the message to that topic by sending a publish message to the broker. The broker then forwards the message to all the clients that are subscribed to that topic, so they can receive the message.

  A publish message has the following format:

  - Fixed Header:
    - *Header* (1 Byte):
      - ◇ Message Type (4 bits): Indicates the type of the MQTT message.
      - ◇ DUP Flag (1 bit): Used to indicate if the message is a duplicate of a previously sent message. It is used depending on the QoS.
      - ◇ QoS Level (2 bits): Indicates the Quality of Service level for the message, which determines the guarantee of delivery for the message.
      - ◇ Retain Flag (1 bit): Used to indicate if the message should be retained by the broker.
    - *Message Length* (1-4 Bytes): This field indicates the length of the payload of the MQTT message, which is the part of the message that contains the actual data being published.
  - Variable Header: This part of the message is optional and its content depends on the type of the MQTT message. It is used for the properties.
  - Topic Name: This field indicates the topic to which the message is being published.
  - Payload: This field contains the actual data being published in the message. At first it had to be a string, but in the latest versions of MQTT it has more options:
    - The Payload Format can be used to define the format of the payload, so it can be an undefined byte array or a UTF-8 encoded value.
    - If UTF-8 encoded value is used, the Content Type property can be used to indicate the type of the content of the payload, so it can be a JSON, XML, etc.

    This makes MQTT very flexible, as it can be used for a wide range of applications with different types of data being published.

Regarding topics, they are just strings that are used to categorize the messages being published. They are organized in a hierarchical structure, where the levels of the hierarchy are separated by a forward slash "/". For instance, a topic could be `home/living_room/temperature`. There are some particularities about topics:

- Wildcards:

  When publishing a message, the topic to which the message is being published must be specified. However, when subscribing to a topic, wildcards can be used to subscribe to multiple topics at the same time. There are two types of wildcards:

- *Single-Level Wildcard* ("`+`"): This wildcard can be used to match any single level in the topic hierarchy. For instance, if a client subscribes to the topic `home/+/temperature`, it will receive messages published to any topic that matches that pattern, such as `home/living_room/temperature`, `home/kitchen/temperature`, etc.
- *Multi-Level Wildcard* ("`#`"): This wildcard can be used to match any number of levels in the topic hierarchy, including the parent level. For instance, if a client subscribes to the topic `home/#`, it will receive messages published to any topic that starts with `home/`, such as `home/garage/door`, `home/living_room/temperature`, `home/kitchen/humidity`, etc.

- Topic Aliases:

  When publishing a message, the topic to which the message is being published must be specified. However, it is common that the same topic is used for publishing multiple messages, so it would be inefficient to include the topic name in every message. To solve this issue, MQTT provides a mechanism for using topic aliases, which allows the publisher to assign an alias to a topic and use that alias instead of the full topic name in subsequent messages. This can significantly reduce the size of the messages being published, especially when the topic names are long. The process for using topic aliases is as follows:

  - The publisher sends a message with the full topic name and a topic alias in the variable header. The topic alias is a unique integer identifier that the publisher assigns to the topic.
  - Both the broker and the subscriber receive the message and store the mapping between the topic alias and the full topic name.
  - In following messages, the publisher can use the topic alias and not include the full topic name, and both the broker and the subscriber can use the stored mapping to determine the full topic name for those messages. The subscriber will add the topic to the message before delivering it to the application.

Once the basic concepts of MQTT have been covered, in the following sections we will cover some of the additional features of MQTT, such as the QoS levels, the retain flag and the last will and testament.

3.4.1. Shared Subscriptions

Once important problem about MQTT is the scalability. If there are multiple subscribers to the same topic, all of them will receive all the messages published to that topic. To solve this issue, MQTT provides a mechanism for shared subscriptions, which allows multiple subscribers to share the same subscription to a topic, so that only one of them receives each message published to that topic. This is specially useful to balance the client-side workload. The process for using shared subscriptions is as follows:

- When a client wants to subscribe to a topic using a shared subscriptions it sends a subscribe message with a special topic format. That format is

`$share/<group_name>/<topic>`, where `<group_name>` is a unique identifier for the group of subscribers sharing the subscription, and `<topic>` is the topic to which they are subscribing.

- The broker receives the subscribe message and adds the client to the group of subscribers for that topic. The broker then ensures that only one client in the group receives each message published to that topic, so it distributes the messages among the clients in the group in a round-robin fashion or using any other load-balancing algorithm.

It should be noted that, for a same topic, there can be both shared and non-shared subscriptions. In that case, the messages published to that topic will be delivered to all the clients with non-shared subscriptions, and to only one of the clients of each shared subscription.

3.4.2. Permanent Sessions

When a client subscribes to a topic, it can choose to have a permanent session or not.

- Permanent Session: If a client chooses to have a permanent session, the broker will queue the subscription information for that client even if the client goes offline, until it goes online again. This means that, if there are messages published to the subscribed topics while the client is offline, those messages will be delivered to the client when it goes online again.

  This is useful for clients that need to receive all the messages published to the subscribed topics, even if they are not always online.

- Non-Permanent Session: If a client chooses to have a non-permanent session, the broker will not queue the subscription information for that client when it goes offline. This means that, if there are messages published to the subscribed topics while the client is offline, those messages will not be delivered to the client when it goes online again.

3.4.3. Request / Response Pattern

MQTT is based on a publish-subscribe communication model, which is different from the traditional request-response communication model used in protocols like HTTP. That lead to users implementing their own request-response pattern on top of MQTT, which is not really efficient. To solve this issue, MQTT provided a mechanism for implementing a request-response pattern, which allows clients to send requests and receive responses using MQTT messages. The process for implementing a request-response pattern is as follows:

- The requester client sends a request. It is simply a publish message with the following properties set in the variable header:

  - *Response Topic*: This property indicates the topic to which the response should be published. The requester client will also subscribe to that topic to receive the response.

- *Correlation Data* (optional): This property can be used as a unique identifier for the request, so the requester client can match the response with the corresponding request when it receives the response.

The topic here is also important, as the clients subscribing to that topic will receive the request.

- The responder client (the one that was subscribed to the topic of the request) receives the request and processes it. Once it has the response ready, it sends a response by publishing a message to the topic specified in the Response Topic property of the request message, and it can also include the Correlation Data property in the response message to allow the requester client to match the response with the corresponding request.

- After receiving the response, the requester client usually unsubscribes from the response topic, as it is not interested in receiving any more responses for that request.

3.4.4. Retain Flag

When a message is published with the retain flag set, the broker will store it (overwriting any previous retained message for that topic). It will then be sent to any client that subscribes to that topic, so it will receive the last retained message for that topic when it subscribes. This has several use cases, such as:

- When the publisher only sends a message when there is a change in the value of the data being published, so the subscribers can always have the last value of that data by receiving the retained message when they subscribe.

- When the publisher sends messages at a very low rate, so the subscribers can receive the last message when they subscribe, even if there are no new messages being published for a long time.

- When a new subscriber joins the network, it can receive a description of each of the clients available in the network. Each client can publish a retained message with a description of itself to `devices/<device_id>`, and the new subscriber can subscribe to `devices/#` to receive the description of all the clients in the network.

3.4.5. Last Will and Testament

When a client disconnects from the broker, there are two different options:

- Graceful Disconnection: The client sends a disconnect message to the broker before disconnecting, so the broker knows that the client has disconnected and can take appropriate actions (as removing the client from the list of subscribers for the topics it was subscribed to). In this case, another action is really common:

  - Before disconnecting, the client can publish a retained message to a specific topic (as `devices/<device_id>`), so the other clients can be notified that that client has disconnected and can take appropriate actions (as considering that client as offline).

- Ungraceful Disconnection: The client disconnects without sending a disconnect message to the broker (maybe because of a network loss), so the broker does not know that the client has disconnected and may still consider it as connected. The broker can detect that the client has disconnected by using a keep-alive mechanism, and after taking the appropriate actions, it can publish a *Last Will and Testament* message.

  A Last Will and Testament (LWT) message is a normal (or retained, if specified) publish message that is specified by the client when it connects to the broker. When the broker detects that the client has disconnected ungracefully[8], it publishes the LWT message to the topic specified in the LWT message, so the other clients can be notified that that client has disconnected and can take appropriate actions (as considering that client as offline).

3.4.6. Quality of Service Levels

MQTT provides three different Quality of Service (QoS) levels for message delivery, which determine the guarantee of delivery for the messages being published. The QoS levels are as follows:

- QoS 0 (At most once): Messages published with QoS 0 are delivered at most once, which means that they may be lost if there is a failure in the network or in the broker.

- QoS 1 (At least once): Messages published with QoS 1 are delivered at least once, which means that they will be delivered even if there is a failure in the network or in the broker. However, if they were sent multiple times due to a failure, they may be delivered multiple times to the subscribers.

- QoS 2 (Exactly once): Messages published with QoS 2 are delivered exactly once, which means that they will be delivered even if there is a failure in the network or in the broker. In addition, they will be delivered only once to the subscribers. This is the highest level of QoS.

The QoS level is defined both in the subscribe message and in the publish message.

- When a client publishes a message, it specifies the QoS level for that message in the publish message. The QoS defines the behavior of the broker, sender and receiver for that message.

- When a client subscribes to a topic, it specifies the maximum QoS level that it wants to receive for messages published to that topic in the subscribe message.

[8] It should be noted that, even when a LWT message had been specified, in graceful disconnections it will not be sent.

The broker then ensures that the messages published to that topic are delivered to that client with a QoS level that is less than or equal to the maximum QoS level specified by the client in the subscribe message.

If a message is published with a QoS level that is higher than the maximum QoS level specified by a subscriber, the broker will deliver that message to that subscriber with the maximum QoS level specified by that subscriber. This allows the subscribers to downgrade the QoS level of the messages they receive if they are not able to handle the higher QoS level.

4. IoT Connected Data

In Section 2.2.1, we identified three main SW functional blocks that are required to implement an IoT application: IoT Networking, IoT Data Management, and IoT Data Processing. The first SW Block has been covered in the previous chapter and, in this chapter, we will focus on the other two SW Blocks, which are closely related to each other and are responsible for handling the data collected from the IoT Things.

In order to manage the data, three aspects are needed:

- Data Models: models that define the structure and semantics of the data collected from the IoT Things.
- Data Storage: mechanisms to store the data collected from the IoT Things, which can be either on-premises or in the cloud.
- Updating Protocols: protocols to update the data collected from the IoT Things in a way that the rest of the system can access it in a timely manner.

4.1. IoT Updating Protocols

Smart Things are usually energy-constrained devices that need to optimize their energy consumption in order to prolong their battery life. Even though the Networking Protocols covered in the previous chapter are designed to be energy-efficient, there are still more energy-saving techniques that can be applied to the data management aspect of IoT applications. There are two main techniques:

- Stream Compression: given a stream of data collected from an IoT Thing, it is possible to apply compression techniques to reduce the amount of data that needs to be transmitted over the network, which can save energy.
- Update Protocols: given a required accuracy of a measurement, it is possible to apply update protocols to reduce the number of measurements needed (and thus the amount of data that needs to be transmitted over the network).

In this section, we will focus on the second technique. Update protocols are designed to reduce the number of measurements needed to achieve a certain level of accuracy, and are usually applicable for data that changes continuously over time, such as temperature, humidity, or location.

Regarding **location data**, a lot of research has been done on update protocols for location data, which is a very common type of data collected from IoT Things. During the next sections, we will cover some of the most common update protocols for location data, which can be applied to other types of data as well.

4.1.1. Periodic Reporting

Periodic reporting is the simplest update protocol, where the IoT Thing reports its location at regular intervals, regardless of whether the location has changed or not. A fixed period T is fixed, and the IoT Thing reports its location every T seconds. The key aspect here is how to choose the period T.

- If T is too small, the IoT Thing will report its location too frequently, which can lead to unnecessary energy consumption and network congestion.
- If T is too large, the IoT Thing will report its location too infrequently, which can lead to inaccurate location data and a poor user experience.

In order to choose the period T, we follow a worst-case approach, where we assume that the IoT Thing is moving in a stright line at its maximum speed $v_{\text{máx}}$, and we want to ensure that the location data is accurate within a certain distance $d_{\text{máx}}$, usually called *accuracy radius* and defined by the application requirements. In this case, we can derive the period T as follows:

$$T = \frac{d_{\text{máx}}}{v_{\text{máx}}}$$

This could however load to unnecessary measurements when:

- The IoT Thing is moving at a speed lower than $v_{\text{máx}}$.
- The IoT Thing is doing micro-movements within the accuracy radius $d_{\text{máx}}$, which can be the case for example for a person sitting on a chair and moving slightly.

4.1.2. Distance-based Reporting

The distance-based reporting protocol is designed to address the limitations of periodic reporting by allowing the IoT Thing to report its location as late as possible, while still ensuring that the location data is accurate within a certain distance $d_{\text{máx}}$. If the last reported location is L_{last}, the IoT Thing will report its location only when the distance between its current location L_{current} and the last reported location L_{last} exceeds the accuracy radius $d_{\text{máx}}$, i.e., when:

$$\text{distance}(L_{\text{current}}, L_{\text{last}}) \geqslant d_{\text{máx}}$$

This protocol allows the send over the internet only the necessary measurements (so it reduces all the possible overhead because of this), but it however needs to measure the location in between the updates. In order to sleep as much as possible, the same worst-case approach of periodic reporting can be applied. Let's assume that:

- The IoT Thing is moving in a straight line at its maximum speed $v_{\text{máx}}$.
- The maximum accuracy radius is $d_{\text{máx}}$.
- The last reported location is L_{last}.

- The measured location of the IoT Thing at time t is $L(t)$.

In that case, at time t, the IoT device can sleep until for T_t seconds, where T_t is defined as follows:

$$T_t = \frac{d_{\text{máx}} - \text{distance}(L(t), L_{\text{last}})}{v_{\text{máx}}}$$

This approach means that, if the IoT Thing is in fact not moving, it will have constant sleep time, but if it is moving towards the accuracy radius, it will have a decreasing sleep time, which allows it to report its location as soon as possible when it reaches the accuracy radius. This has however some problems:

- As the IoT Thing reaches the accuracy radius, it will have to measure its location more and more frequently, making it not energy-efficient. Therefore, a minimum sleep time $T_{\text{mín}}$ can be defined, which is the minimum time that the IoT Thing can sleep before measuring its location again.
- Measuring the location of the IoT Thing may also take some time (t_{measure}). If the calculated sleep time T_t is less than the measurement time t_{measure}, the IoT Thing will not be able to sleep for the calculated time, and it will have to measure its location immediately.

Therefore, in order to address these problems, instead of sending the location data only when the accuracy radius is reached (where the sleep time T_t would be zero), the IoT Thing will send the location data when:

$$T_t \leqslant \text{máx}\{T_{\text{mín}}, t_{\text{measure}}\}$$

Even though this last modification allows the IoT Thing to sleep for a longer time, it still requires the IoT Thing to measure its location several times before actually sending the location data, which can lead to unnecessary energy consumption. The next protocol will address this problem.

4.1.3. Early Distance-based Reporting

The early distance-based reporting protocol is designed to address the limitations of the distance-based reporting protocol by allowing the IoT Thing to report its location *before* reaching the accuracy radius, which can lead to longer sleep times and thus more energy savings. That way, a balance can be reached between:

- The energy consumption for sending the location data over the network.
- The energy consumption for measuring the location of the IoT Thing.

The idea used to reach that balance is called the *Energy-based update condition*. It compares two energy consumption values:

- The energy consumption for sending the location data over the network inmediately.
- The energy consumption for dropping the current measurement and waiting for the next measurement (where we will again apply this comparison). The waited time is calculated as in the Distance-based reporting protocol.

The IoT will always choose the option with the lowest energy consumption, which allows it to save energy while still ensuring that the location data is accurate within a certain distance $d_{\text{máx}}$. However, in order to effectively apply this protocol, the IoT Thing needs to have a good estimation of the energy consumption for both options (which depends on the future location of the IoT Thing, which is not known). Therefore, some heuristics are applied. We will explore the two main heuristics used for this purpose.

Next Fix heuristic

The next fix heuristic is based on the idea of stimating the energy consumption for both options.

- The energy consumption for sending the location data over the network inmediately is estimated as follows:

$$E_{\text{update}} = \frac{E_{\text{send}} + E_{\text{measure}}}{T_0}$$

  Where T_0 is the time until the next measurement when an update is sent ($t = 0$).

- The energy consumption for dropping the current measurement and waiting for the next measurement is estimated as follows:

$$E_{\text{no-update}} = \frac{E_{\text{measure}}}{T_t}$$

  Where T_t is the time until the next measurement when no update is sent (calculated as in the Distance-based reporting protocol).

Depending on the values of E_{update} and $E_{\text{no-update}}$, the IoT Thing will choose the option with the lowest energy consumption.

Predicted Movement heuristic

This heuristic is much more complex than the Next Fix heuristic, as it is based on the idea of predicting the future location of the IoT Thing in order to estimate the energy consumption for both options more accurately. It divides the time in *cycles*, and:

- Updating the location data over the network starts a new cycle.
- Dropping the current measurement and waiting for the next measurement continues the current cycle.

4.1.4. Dead Reckoning

Dead reckoning is a similar technique to the Predicted Movement heuristic, as it allows the IoT Thing to estimate its location based on its previous location and its movement. It works as follows:

- Initially, the IoT Thing reports its current state (location) and an expected update function (which is a function that estimates the future location of the IoT Thing based on its current location and its movement). That update function may be very complex:
  - A linea movement can be estimated, but it may be too simple for some applications.
  - A more complex movement can be estimated using machine learning techniques. However, this requires more computational resources on the IoT Thing, which can lead to higher energy consumption. This is a trade-off that needs to be carefully considered when designing the expected update function.
- After that, both the cloud server and the IoT Thing have that function that can be used to estimate the location of the IoT Thing in the future.
- Periodically, the IoT Thing measures its location and compares it with the estimated location. If the error between the measured location and the estimated location exceeds a certain threshold, the IoT Thing reports its current state (location) and an new updated expected update function.

That way, the IoT Thing can sleep for a longer time, and it only needs to report its location when the error between the estimated location and the measured location exceeds a certain threshold, which can lead to significant energy savings. Meanwhile, the cloud server can use the expected update function to estimate the location of the IoT Thing at any time, which allows it to have accurate location data without needing to receive updates from the IoT Thing too frequently.

5. Question Sheets

5.1. Introduction

The recorded lecture is provided here.

Ejercicio 5.1.1. What are the two views on IoT discussed in the lecture? How do they differ?

The two views on IoT are:

- IoT as a network of connected *things*, aka *M2M (Machine to Machine)*.
- IoT as a network of connected *data*, aka *Big Data*.

They differ in plenty of different aspects:

- While the first one focuses on the devices and the communication between them, the second one focuses on the data that is generated by these things and how it can be used to create value.
- While the first one sees Internet as just a communication medium, the second one sees Internet as a platform for data collection and analysis.
- While the main goal of the first one is to connect as many things as possible, the main goal of the second one is to extract insights from the data.
- While the first one is usually taken by engineers and computer scientists, the second one is usually taken by business people and data scientists.

Ejercicio 5.1.2. Are IoT gateways always necessary? What are they used for?

IoT gateways are used to connect IoT things to the internet. IoT things are usually resource-constrained devices that cannot connect directly to the internet, so they need a gateway to act as a bridge between them and the internet. However, IoT gateways are not always necessary, as some IoT things can connect directly to the internet using low-power communication technologies such as Wi-Fi or cellular. In this case, the IoT thing itself can act as a gateway, and there is no need for an additional device.

5.2. IoT Systems

The recorded lecture is provided here.

Ejercicio 5.2.1. What is the difference between an IoT system and an IoT platform? Use the WWW to find another IoT platform that is not mentioned in the lecture.

They are different concepts. An IoT system consists of all the HW and SW components that are required to implement an IoT application, and developpers tend to reuse the same components across different applications. This is where the concept of IoT platforms comes into play, because an IoT platform is a framework that provides a set of tools and services to facilitate the development, deployment, and management of IoT applications. By using an IoT platform, developers can focus on the application logic rather than dealing with the complexities of the underlying infrastructure. Therefore, an IoT platform is a tool that can be used to build an IoT system.

An example of an IoT platform that is not mentioned in the lecture is Blynk. It is an horizontal IoT platform that provides a set of tools and services to facilitate the development, deployment, and management of IoT applications. It allows developers to create custom IoT applications using a drag-and-drop interface, and it supports a wide range of hardware platforms and communication protocols.

Ejercicio 5.2.2. Is the Microsoft Azure IoT Suite a horizontal or a vertical IoT platform? Why?

It is an horizontal IoT platform because it provides a broad set of features and services that can be customized to fit the specific needs of different applications. It is not designed for a specific industry or use case, but rather it can be used for a wide range of IoT applications across different domains.

Ejercicio 5.2.3. Provide an example (i.e., search on the Internet or come up with your own idea) for a scenario in which you would use predictive data processing but not prescriptive data processing. What is the main legal difference between these?

There are plenty of cases where predictive data processing can be used without prescriptive data processing for several reasons:

- Prescriptive data processing can simply not be made.

  For instance, an IoT application designed to predict the weather conditions in a specific area based on historical data and patterns. The application can provide valuable insights and information about the expected weather conditions, but the weather cannot be changed by the application, so prescriptive data processing is not possible in this case.

- Prescriptive data processing can be made, but it is not ethical to do it without human supervision and intervention.

  For instance, an IoT application designed to predict the likelihood of a patient developing a certain medical condition based on their health data and lifestyle

habits. The application may recommend certain lifestyle changes or medical interventions to reduce the risk of developing the condition, but it is not ethical to implement these recommendations without human supervision.

- Prescriptive data processing can be made, but a responsible person must be legally accountable for the consequences of the actions taken based on the prescriptive data processing.

  For instance, an IoT application designed to predict the future of the stock market based on historical data and patterns. The application may recommend certain investment strategies to maximize returns, but someone should be legally accountable for the consequences of the actions taken based on these recommendations, as the stock market is highly volatile and unpredictable, and there is a risk of financial loss.

The main legal difference between predictive and prescriptive data processing is that prescriptive data processing actually performs actions or makes decisions, and they can have legal consequences, so there must be a responsible person who is legally accountable for those consequences.

Ejercicio 5.2.4. What are the differences between mesh-based and edge-based IoT systems? Which one is currently supported by platform providers?

Both IoT system architectures are designed to process data closer to where it is generated, but in the edge-based architecture the IoT things are still really simple and resource-constrained, so they just collect data and send it to the gateway, which is responsible for processing the data and making decisions based on it. In contrast, in the mesh-based architecture, the IoT things are more powerful and can communicate directly with each other, so they can process data and make decisions without the need for a central gateway.

Currently, even though the most supported architecture by platform providers is the cloud-based one, there are some platforms that support edge-based IoT systems, but mesh-based IoT systems are still not widely supported by platform providers due to the complexity and challenges associated with implementing and managing such systems.

5.3. IoT Internetworking

5.3.1. Motivation

The recorded lecture is provided here.

Ejercicio 5.3.1. Why do we need new protocols for the IoT instead of using established Internet protocols? Describe one challenge existing protocols face.

New protocols are needed for the IoT because existing Internet protocols, such as TCP/IP, are not optimized for the resource-constrained nature of IoT devices. One challenge existing protocols face is that they may require larger buffer sizes than what is available on IoT devices, which can lead to performance issues and increased latency.

Ejercicio 5.3.2. Which central protocol can we not replace (completely) for the IoT? Why is it harder to replace it than others?

The central protocol that we cannot replace (completely) for the IoT is the Internet Protocol (IP). It is harder to replace IP than other protocols because it is the fundamental protocol that enables communication and data exchange between different devices and applications on the Internet. It is the backbone of the Internet and, technically speaking, being connected to the Internet requires having an IP address and using IP for communication. Therefore, while we can use different protocols for the application layer or the transport layer, we still need to use IP at the network layer in order to enable connectivity and communication with the Internet. In addition, replacing IP would require a complete overhaul of the Internet infrastructure and would require all devices and applications to adopt the new protocol, which is a significant challenge in terms of compatibility and interoperability.

Ejercicio 5.3.3. We introduced two approaches for Smart Things to connect to the Internet. Name these approaches, describe them briefly and give one advantage and one disadvantage of each, compared to the other.

The two approaches for Smart Things to connect to the Internet are:

- Application layer proxy: In this approach, there is a proxy (usually implemented in the IoT gateway) that translates between the protocols used by the IoT Things and the protocols used by the Internet. The IoT Thing communicates with the proxy using dedicated IoT protocols that have nothing to do with the Internet protocols, and the proxy stores the data received from the IoT Things and makes it available to the Internet using standard Internet protocols.
    - Advantage: It allows for the use of dedicated IoT protocols that are optimized for the resource-constrained nature of IoT devices, which can lead to better performance and energy efficiency.
    - Disadvantage: It introduces an additional layer of complexity and can lead to increased latency and reduced scalability (due to the dedicated software), as all communication between the IoT Things and the Internet needs to go through the proxy.

- IP Router: In this approach, the IoT Things directly use IP, but with some optimizations and adaptations to address the challenges and limitations mentioned above. There is no need for specific apps or software to interact with the IoT Things, as they can be accessed using standard Internet protocols and interfaces.
    - Advantage: It provides better scalability and interoperability, as the IoT Things can communicate directly with other devices and applications on the Internet without the need for a proxy.
    - Disadvantage: It may require more complex and resource-intensive protocols and technologies to enable efficient communication and data exchange between the IoT Things and the Internet, which can lead to increased power consumption and reduced performance for resource-constrained IoT devices.

5.3.2. IEEE 802.15.4

The recorded lectures are provided here (Part A) and here (Part B).

Ejercicio 5.3.4. What are the differences between the PHY and the MAC layer?

Both are layers in the OSI model, but they serve different purposes. The PHY (Physical) layer is responsible for the physical transmission of data over the communication medium. The MAC (Medium Access Control) layer builds on top of the PHY layer and is responsible for managing access to the communication medium, including addressing, framing, and error detection.

Ejercicio 5.3.5. Discuss pros and cons of using 16 bit addresses for 802.15.4 instead of 64 bit addresses.

Some pros of using 16 bit addresses for 802.15.4 include:

- Reduced overhead: 16 bit addresses require less memory and bandwidth compared to 64 bit addresses.
- Faster processing: 16 bit addresses can be processed more quickly than 64 bit addresses, which can lead to improved performance and reduced latency.

Some cons of using 16 bit addresses for 802.15.4 include:

- Limited address space: 16 bit addresses can only support up to $2^{16} = 65536$ unique addresses, which may not be sufficient for large-scale IoT deployments.
- Not globally unique: 16 bit addresses are not globally unique, so they can only be used within a local network, which can limit interoperability and scalability.
- Harder to manage: They have to be managed and assigned manually, which can lead to errors and conflicts in larger networks.
- If a device moves to another network, it may need to change its address, which can lead to additional overhead and complexity in managing the network.

Ejercicio 5.3.6. Describe the basic idea of CSMA/CA and compare it to frequency hopping. You may use a diagram / drawing to describe your answer.

CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) is a protocol used in wireless communication to manage access to the communication medium. It works by having devices listen to the channel before transmitting data. If the channel is busy, the device will wait for a random backoff time before trying again. This helps to avoid collisions and improve the efficiency of the network. In contrast, frequency hopping is a technique where devices rapidly switch between different frequency channels to avoid interference and improve security. The hopping pattern is typically determined by a pseudo-random sequence, so no two devices will hop to the same frequency at the same time, which means that they can transmit simultaneously without interference.

Ejercicio 5.3.7. Briefly describe and compare TDMA and FDMA. Which one is better?

TDMA (Time Division Multiple Access) is a protocol that divides the communication medium into time slots, and each device is assigned the whole frequency channel for a specific time slot. This means that, while only one device can transmit at a time, it can use the full bandwidth of the channel during its assigned time slot. FDMA (Frequency Division Multiple Access) is a protocol that divides the communication medium into frequency channels, and each device is assigned a specific frequency channel for transmission. This means that multiple devices can transmit simultaneously, but they have to share the bandwidth of the channel. The choice between TDMA and FDMA depends on the specific requirements of the application and the characteristics of the communication medium.

- Depending on the application.

  TDMA may be better for applications that require high data rates but can tolerate some delay, while FDMA may be better for applications that require no delay but can tolerate lower data rates.

- Depending on the channel characteristics.

  If the bandwidth of the channel is limited, TDMA may be more efficient as it allows devices to use the full bandwidth during their assigned time slots. On the other hand, if the channel has a large bandwidth and can support multiple simultaneous transmissions, FDMA may be more efficient as it allows multiple devices to transmit at the same time without interference.

- Depending on the number of devices.

  If there are a large number of devices that need to access the communication medium, FDMA may be more efficient as it allows multiple devices to transmit simultaneously, while TDMA may lead to increased latency and reduced performance due to the need for devices to wait for their assigned time slots.

Ejercicio 5.3.8. An 802.15.4 network is deployed in the following three application scenarios. Which CSMA/CA MAC Layer version would you use? Justify your answer (e.g., "In this case, we should use slotted CSMA/CA, because...").

- A smart home network, where only 10 temperature sensors are installed. Each sensor reports a measurement once per minute.

  Given the low number of devices, the low data rate, and the fact that the devices are not synchronized, we can use unslotted CSMA/CA, as it is simpler and more efficient for this scenario.

- A smart clinic network that transmits monitoring data about the patients and alarm notifications.

  In this case, we should use slotted CSMA/CA, because it can provide better performance and reduced latency for time-sensitive applications, such as monitoring data. In addition, thanks to the GTS mechanism, it can provide guaranteed time slots for alarm notifications, which can be critical for patient safety.

Ejercicio 5.3.9. When unslotted CSMA/CA is applied, at each trial the back-off window size will be doubled. However, it doesn't always lead to a longer delay, why?

The back-off window size is doubled after each failed transmission attempt, but this does not always lead to a longer delay because the back-off time is randomly selected from the back-off window. The expected back-off time may be doubled, but it is possible that the randomly selected back-off time is shorter than the previous one, which can lead to a shorter delay.

5.3.3. NB-IoT

The recorded lectures are provided here (Part A) and here (Part B).

Ejercicio 5.3.10. Explain the basic architecture of a cellular network. How does it cover large areas?

A cellular network is a wireless communication network that is divided into smaller geographic areas called cells. Each cell is served by a base station, which is responsible for providing wireless coverage and connectivity to the devices within its cell. The base stations are connected to a central network infrastructure, which manages the communication and data exchange between the devices and the Internet. The cellular network covers large areas by using multiple cells that are strategically placed to provide coverage to the desired geographic area. The cells can be of different sizes, depending on the population density and the demand for wireless services in the area. By using multiple cells, the cellular network can provide coverage to a large number of devices while maintaining good performance and reliability.

Ejercicio 5.3.11. Briefly describe an application case in which NB-IoT is better than IEEE 802.15.4. Justify your answer.

An application case in which NB-IoT is better than IEEE 802.15.4 is whenever you need to cover a large area with a low density of devices, such as in smart agriculture or smart city applications. NB-IoT is designed for wide-area coverage and can support a large number of devices over a wide geographic area, while IEEE 802.15.4 is designed for short-range communication and is better suited for applications that require low power consumption and low data rates. However, IEEE 802.15.4 should not be forgotten, as it works in the unlicensed spectrum and can be used without the need for a cellular network infrastructure, which can be advantageous in certain scenarios, such as in remote areas or in applications that require low-cost deployment.

Ejercicio 5.3.12. Name and briefly explain two of the possible deployment scenarios that specify the frequency range that NB-IoT may be deployed in.

Two possible scenarios for NB-IoT deployment are:

- In-band deployment: In this scenario, NB-IoT is deployed within the existing LTE spectrum, using the same frequency bands as LTE. This allows for easy integration with existing LTE infrastructure and can provide good performance and coverage for NB-IoT devices. However, it may lead to increased interference and reduced performance for both NB-IoT and LTE devices, as they are sharing the same frequency bands.

- Guard-band deployment: In this scenario, NB-IoT is deployed in the guard bands between LTE frequency bands. This allows for better isolation and reduced interference between NB-IoT and LTE devices, as they are using different frequency bands. However, it may require additional spectrum allocation and may not be as widely supported as in-band deployment, which can limit the availability and coverage of NB-IoT devices.

Ejercicio 5.3.13. How are frequency and time multiplexing used in NB-IoT?

NB-IoT uses both frequency and time multiplexing to enable efficient communication and data exchange between the devices and the base station.

- Frequency multiplexing: NB-IoT uses frequency division multiplexing (FDM) to divide the available frequency spectrum into two different types of channels: uplink channels for transmitting data from the devices to the base station, and downlink channels for transmitting data from the base station to the devices.
- Time multiplexing: NB-IoT uses time division multiplexing (TDM) to divide the physical channel into time slots, and each time slot is assigned to a specific logical channel. This allows multiple devices to share the same frequency channel by transmitting and receiving data in different time slots.

Ejercicio 5.3.14. Explain the procedure that a newly powered on user equipment (UE) follows to join a cell.

When a newly powered on user equipment (UE) wants to join a cell, it follows the following procedure:

- The UE listens in a broadcast channel for a signal from base stations, which contains information about the available cell and its characteristics.
- The UE checks if the cell is joinable, as some cells may be reserved for specific types of devices or applications.
- If the cell is joinable, the UE checks if it has enough signal strength to connect to the cell.
- Among the cells that are joinable and have enough signal strength, the UE selects the one with the best signal quality (e.g., the one with the highest signal-to-noise ratio) and sends a connection request to the base station of that cell.

Ejercicio 5.3.15. What is a tracking area in NB-IoT and how is it used?

A tracking area in NB-IoT is a group of cells that share the same tracking area identifier (TAI). The MME (Mobility Management Entity) keeps track of the tracking area that is serving the UE, so when there is data waiting for the UE, only the cells in that tracking area are paged. When the UE needs to inform the MME of the cell that is serving it, it sends a Tracking Area Update (TAU) message to the MME, which updates in its internal records the tracking area that is serving the UE. This allows for a trade-off between the overhead of paging and the overhead of tracking, as it reduces the overhead of paging while still allowing for efficient tracking of the UE's location within the network.

Ejercicio 5.3.16. How can a UE use Extended Discontinuous Reception to save energy?

Extended Discontinuous Reception (eDRX) is a mode that allows the UE to save energy by sleeping not just between the start of each hyperframe (when paging is performed), but for a configurable number of hyperframes. This means that the UE can configure the number of hyperframes that it will sleep for, so it will only listen for paging messages at the start of each hyperframe after sleeping for that number of hyperframes. This allows the UE to save energy by sleeping for longer periods of time, while still being able to receive paging messages from the base station when needed. During all that time, the SGW retains the incoming data for the UE in buffers.

5.3.4. 6LoWPAN

The recorded lectures are provided here (Part A) and here (Part B).

Ejercicio 5.3.17. Explain why we need 6LoWPAN to support IPv6 over IEEE 802.15.4. Also explain briefly how 6LoWPAN can make IPv6 over IEEE 802.15.4 more efficient.

We need 6LoWPAN to support IPv6 over IEEE 802.15.4 strictly just because of one reason: IP requires a minimum MTU of 1280 bytes, while IEEE 802.15.4 has a maximum frame size of 127 bytes. This means that we cannot directly use IPv6 over IEEE 802.15.4 without some form of adaptation or optimization, as the IPv6 packets would be too large to fit within the frame size of IEEE 802.15.4. 6LoWPAN provides fragmentation in order to transparently split IPv6 packets into smaller fragments that can fit within the frame size of IEEE 802.15.4.

However, in order to make IPv6 over IEEE 802.15.4 more efficient, 6LoWPAN also provides two main optimizations:

- Header compression: 6LoWPAN provides header compression techniques that can significantly reduce the size of the IPv6 header, which can increase the payload rate and reduce the overhead of transmitting IPv6 packets over IEEE 802.15.4.
- Stateless Address Autoconfiguration: 6LoWPAN provides a mechanism for stateless address autoconfiguration, which allows devices to automatically generate their own IPv6 addresses based on their MAC addresses and the LL prefix. This can simplify the configuration and management of IPv6 addresses for devices in an IEEE 802.15.4 network, and can also reduce the overhead of address management and configuration.

Ejercicio 5.3.18. What is an Encapsulation Header Stack in 6LoWPAN?

An Encapsulation Header Stack in 6LoWPAN is a stack of headers that are added to the IPv6 packet in order to provide additional information and functionality for the transmission of the packet over IEEE 802.15.4. The Encapsulation Header Stack can include headers for fragmentation and header compression, as well as other headers that may be needed for specific applications or use cases. The Encapsulation Header Stack is added to the IPv6 packet before it is transmitted over IEEE 802.15.4, and it is removed at the receiving end before the IPv6 packet is processed by the upper layers of the protocol stack.

Ejercicio 5.3.19. Explain the concept of "Stateless Address Autoconfiguration" for 6LoWPAN, without going into detail about how an address is created.

Stateless Address Autoconfiguration for 6LoWPAN is a mechanism that allows devices to automatically generate their own IPv6 addresses without the need for a central address management system, as DHCP does. The addresses generated by this mechanism are however link-local addresses, which means that they can only be used for communication within the same link (e.g., within the same IEEE 802.15.4 network), and they cannot be used for communication with devices outside of the link (e.g., with devices on the Internet). This mechanism is particularly useful for resource-constrained devices in an IEEE 802.15.4 network, as it allows them

to generate their own addresses without the need for additional configuration or management overhead.

Ejercicio 5.3.20. Explain briefly how fragmentation in 6LowPAN relates to fragmentation in IP. Does it, e.g., replace it?

Given that the maximum frame size of IEEE 802.15.4 is much smaller than the minimum MTU required by IP, fragmentation in 6LoWPAN is necessary to enable the transmission of IPv6 packets over IEEE 802.15.4. However, fragmentation in 6LoWPAN does not replace fragmentation in IP, as the longest possible IPv6 packet that can be fragmented by 6LoWPAN is 1280 bytes, which is still small for the maximum size of an IPv6 packet (which can be up to $2^{16} - 1 = 65535$ bytes). Therefore, if an IPv6 packet is larger than 1280 bytes, it would still need to be fragmented at the IP layer in order to be transmitted over IEEE 802.15.4, and the resulting fragments would then be further fragmented by 6LoWPAN in order to fit within the frame size of IEEE 802.15.4.

Ejercicio 5.3.21. A single IPv6 packet with a size of 555 Byte (including the IPv6 header and its payload) is sent to an IoT device using 802.15.4 and 6LoWPAN. The IPv6 header is uncompressed, and we use 6LoWPAN fragmentation. What would be the value of the datagram offset field in the 5th fragment? Justify your answer.

Observación. We assume each fragment has 72 bytes available for its payload.

1. For the first fragment, given that $72 = 9 \cdot 8$ is divisible by 8, we can use all the 72 bytes for the payload, so the datagram contains bytes 0-71 of the original IPv6 packet. It has no offset, as it is the first fragment.

2. For the second fragment, we can also use all the 72 bytes for the payload, so the datagram contains bytes 72-143 of the original IPv6 packet. The datagram offset is 9, as $^{72}/_{8} = 9$.

3. For the third fragment, we can also use all the 72 bytes for the payload, so the datagram contains bytes 144-215 of the original IPv6 packet. The datagram offset is 18, as $^{144}/_{8} = 18$.

4. For the fourth fragment, we can also use all the 72 bytes for the payload, so the datagram contains bytes 216-287 of the original IPv6 packet. The datagram offset is 27, as $^{216}/_{8} = 27$.

5. For the fifth fragment, we can also use all the 72 bytes for the payload, so the datagram contains bytes 288-359 of the original IPv6 packet. The datagram offset is 36, as $^{288}/_{8} = 36$.

Ejercicio 5.3.22. Explain when using HC1 and HC2 for header compression is especially useful and when it is not very useful.

Using HC1 and HC2 for header compression is especially useful when the IPv6 packets being transmitted over IEEE 802.15.4 have LL auto-configured source and destination addresses, as these addresses can be compressed to two bits each using HC1, which can significantly reduce the size of the IPv6 header and increase the payload rate. In addition, they are useful when the next header is either UDP, ICMP,

or TCP, as these can also be compressed using HC2, which can further reduce the size of the header and increase the payload rate. In other cases, they may not be very useful.

5.3.5. MQTT

The recorded lecture is provided here.

Ejercicio 5.3.23. Given the following MQTT configuration with five clients $C1, C2, C3, C4, C5$ and a broker B, which clients receive published messages in the following scenarios?

(a) The situation is:

1) $C1$ subscribes to topic `home/groundfloor/temp` with the retain flag set to "1".
2) $C2$ subscribes to topic `home/secondfloor/temp` with the retain flag set to "0".
3) $C3$ subscribes to topic `home/thirdfloor/livingroom/temp` with the retain flag set to "1".

After all publications are done, $C4$ subscribes to topic `home/secondfloor/temp` and $C5$ subscribes to topic `home`.

In this case:

- $C4$ would not receive any message, as the retain flag for the message published by C2 was set to "0", which means that the broker does not store the message and does not send it to new subscribers.
- $C5$ would not receive any message, as its topic filter `home` does not match any of the topics that $C1$, $C2$, and $C3$ published messages for.

(b) The situation is:

1) $C1$ subscribes to topic `home/thirdfloor/+/temp`.
2) $C2$ publishes a message for topic `home/groundfloor/temp` with the retain flag set to "1".
3) $C3$ publishes a message for topic `home/secondfloor/temp` with the retain flag set to "0".
4) $C4$ publishes a message for topic `home/thirdfloor/livingroom/temp` with the retain flag set to "0".

After all publications are done, $C5$ subscribes to topic `home/thirdfloor/livingroom/temp`.

In this case:

- $C1$ would receive the message published by $C4$, as its topic filter `home/thirdfloor/+/temp` matches the topic of the message published by $C4$.
- $C5$ would not receive any message, as the retain flag for the message published by $C4$ was set to "0".

(c) The situation is:

1) $C1$ subscribes to topic `home/+/temp`.

2) $C2$ subscribes to topic `home/secondfloor/temp`.

3) $C3$ publishes a message for topic `home/+/temp`.

It is not possible to publish a message for topic `home/+/temp`, as the topic filter `home/+/temp` contains a wildcard character, which is not allowed in the topic of a published message. Therefore, no client would receive any message in this case.

(d) The situation is:

1) $C1$ subscribes to topic `home/#` with QoS level 0.

2) $C2$ subscribes to topic `home/+/temp` with QoS level 0.

3) $C3$ publishes a message for topic `home/thirdfloor/temp` with QoS level 2.

In this case:

- $C1$ would receive the message published by $C3$, as its topic filter `home/#` matches the topic of the message published by $C3$. However, it would receive the message with QoS level 0, as that is the QoS level of its subscription, which means that the broker would not guarantee the delivery of the message to $C1$.
- $C2$ would also receive the message published by $C3$, as its topic filter `home/+/temp` matches the topic of the message published by $C3$. However, it would also receive the message with QoS level 0, as that is the QoS level of its subscription, which means that the broker would not guarantee the delivery of the message to $C2$.

Ejercicio 5.3.24. Client $C2$ subscribes to the topic `a/b/c` with QoS level 0. Afterwards, client $C1$ publishes a message to the topic `a/b/c` with QoS level 2. Looking at the message headers:

1. What is the QoS level between $C1$ and the broker?

   Since $C1$ published the message with QoS level 2, the QoS level between $C1$ and the broker is 2, which means that the broker will guarantee the delivery of the message to $C1$ by using a four-step handshake process.

2. What is the QoS level between the broker and $C2$?

   Since $C2$ subscribed to the topic with QoS level 0, the QoS level between the broker and $C2$ is 0, which means that the broker will not guarantee the delivery of the message to $C2$ and will simply send it without any acknowledgment or retry mechanism.

Ejercicio 5.3.25. Explain how to use "Shared Subscriptions" in MQTT.

Shared Subscriptions in MQTT allow multiple clients to share the same subscription, which means that when a message is published to the topic of the shared subscription, it will be delivered to only one of the clients that are part of the shared subscription. This can be useful for load balancing and scalability, as it allows

multiple clients to share the workload of processing messages for a specific topic. To use Shared Subscriptions, clients can subscribe to a topic using a special syntax that includes a shared subscription identifier (e.g., `$share/group1/topic`), and the broker will manage the distribution of messages among the clients that are part of the shared subscription.

5.4. Connected Data

5.4.1. Update Protocols

The recorded lectures are provided here (Introduction) and here (Update Protocols).

Ejercicio 5.4.1. What is an update protocol and how does it differ from stream compression?

Ejercicio 5.4.2. Explain the update protocol "Periodic Reporting" and describe a situation in which it works very well and a situation in which it does not work well. Which other update protocol can we use to solve the problem?

5.4.2. Data Models

The recorded lecture is provided here.

Ejercicio 5.4.3. Give an example for an information that a Key-Value Model cannot represent but a Linked-data Based Model can.

5.4.3. Stream Processing and Complex Event Processing

The recorded lectures are provided here (Introduction), here (Stream Processing) and here (Complex Event Processing).

Ejercicio 5.4.4. Compare Stream Processing with Complex Event Processing by discussing the different information models, what they each focus on and where they come from.

Ejercicio 5.4.5. What is a continuous query? Give an example for one in pseudo code (i.e., define your own language for continuous queries and provide an example with it). Include typical features like, e.g., a window(-ing) concept into your language.

Ejercicio 5.4.6. What are Event-Condition-Action rules and how do they differ from continuous queries? What is similar?

5.4.4. IoT Machine Learning

The recorded lecture is provided here.

Ejercicio 5.4.7. Why would we want to move machine learning to the edge or even into the smart IoT thing? What are potential advantages? What are challenges and when would you not move it but keep it in the Cloud?

Ejercicio 5.4.8. Briefly explain how magnitude-based pruning is performed.

Ejercicio 5.4.9. Explain the difference between pruning and quantization of an artificial neural network. What is binarization and why has it the potential to be implemented extremely efficiently in hardware?